Package 'gltbnn'

November 11, 2026

Type Package

Title Variational Bayesian Neural Networks with Global-Local Shrinkage Priors

Version 0.2.0

Description Fit Bayesian neural networks for Gaussian regression, binary logistic regression, and multinomial classification with global, horseshoe, or global-local-tail shrinkage on individual hidden weights and biases. The global-local-tail model learns a separate tail shape for each hidden layer. Variational inference yields draws of the regression function, predictive draws for continuous outcomes, and predictive class probabilities. The tail-adaptive prior builds on Lee et al. (2024) <doi:10.1214/24-EJS2317>.

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.1.0)

Imports stats

URL https://yain22.github.io/Bayesian_NN_GLT/, https://github.com/yain22/Bayesian_NN_GLT

BugReports https://github.com/yain22/Bayesian_NN_GLT/issues

NeedsCompilation no

Author Se Yoon Lee [aut, cre, cph]

Maintainer Se Yoon Lee <seyoonlee.stat.math@gmail.com>

Contents

gltbnn_fit 2
gltbnn_shrinkage 5
predict.gltbnn_fit 6

Index **8**

`gltbnn_fit` *Fit a Bayesian neural network with shrinkage priors*

Description

Fit a fully connected neural network for Gaussian regression, binary logistic classification or categorical softmax classification. Each hidden layer uses a rectified linear unit (ReLU) activation.

Usage

```
gltbnn_fit(x, y, hidden = c(10L, 10L), prior = c("GLT", "GL", "G"),
           control = list(), seed = 1L, noise_prior = NULL,
           outcome = c("continuous", "binary", "categorical"),
           geometry = c("coefficient", "unit"))

## S3 method for class 'gltbnn_fit'
print(x, ...)
```

Arguments

`x`	For fitting, a numeric matrix or numeric data frame with one row per observation and one column per predictor. Supply at least two observations and one predictor. For printing, a fitted `gltbnn_fit` object.
`y`	One response per row of `x`. For regression, supply finite numeric responses with a positive sample standard deviation. For classification, supply a factor or finite numeric, logical or nonempty character labels. Every declared factor level must occur in training. Factor levels set the class order. Other label vectors use the level order of `factor(y)`.
`hidden`	Positive integer widths in input-to-output order. For example, `c(4L, 8L)` creates four neurons in the first hidden layer and eight in the second. Biases are included automatically.
`prior`	One of `"G"` (global), `"GL"` (horseshoe), or `"GLT"` (global-local-tail, the default).
`control`	Named list of overrides for the fitting controls described below.
`seed`	Nonnegative integer random seed. The previous `.Random.seed` is restored after fitting.
`noise_prior`	Prior for the observation variance on the standardized response scale in continuous regression. The default `NULL` uses the paper's Jeffreys prior. Use `NULL` for classification. For a proper inverse-gamma prior, supply a numeric vector named `shape` and `scale`, both finite and strictly positive. For example, `c(shape = 3, scale = 2)` has mean 1 in standardized units.
`outcome`	`"continuous"` (the default), `"binary"` for exactly two classes, or `"categorical"` for two or more nominal classes.
`geometry`	`"coefficient"` (the default) gives each hidden weight and bias its own local scale under GL and GLT. Use `"unit"` to fit the earlier grouped specification, which shares one local scale across each unit's incoming coefficients. G has one common scale per layer in both specifications, with different prior calibration as described below.
`...`	Additional arguments to the print method, currently unused.

Details

The incoming weights and bias of neuron k in hidden layer l form a vector $\mathbf{w}_{kl}$ of $d_l = K_{l-1} + 1$ coefficients. Here K_l is the width of layer l, K_0 is the number of predictors used by the network, and $M_l = K_l d_l$ counts its scalar hidden coefficients, including biases. In Gaussian regression, coefficient $w_{kl,j}$ has a zero-centered Gaussian prior with conditional standard deviation $\sigma\lambda_{kl,j}$. The final coordinate $j = d_l$ is the unit's bias. The weights are independent given their scales and the observation variance. The quantity σ^2 is the Gaussian observation noise variance on the standardized training scale. The same σ sets the units for the hidden weight prior. The default uses the improper density kernel $1/\sigma^2$. Its posterior is proper almost surely when the responses have a joint density conditional on the predictors. Some nonconstant data can yield an improper posterior. For example, under G with one hidden layer, three or more distinct univariate inputs and exactly affine responses with nonzero slope can have this property. Interpret variational draws as approximate posterior draws when the posterior is proper.

The alternative inverse-gamma prior gives a proper posterior for every finite dataset accepted by this function. For variance s, shape a and scale b, its density is proportional to $s^{-(a+1)} \exp(-b/s)$. The prior uses standardized fitting units. Its scale parameter on the original response scale equals b times the training sample variance of `y`.

With `"G"`, every coefficient in a layer shares the scale $\lambda_{kl,j} = \tau_l$. With `"GL"`, $\lambda_{kl,j} = \tau_l r_{kl,j}$, where the multipliers are independent standard half-Cauchy variables. Both priors give τ_l a half-Cauchy prior with scale $1/M_l$. With `"GLT"`, the coefficient scales independently follow a generalized Pareto distribution conditional on the layer scale τ_l and tail shape ξ_l. The model learns both for each layer. Given ξ_l, the prior on τ_l is inverse-gamma with shape $1 + M_l/\xi_l$ and scale 1. The prior on $\log \xi_l$ is standard normal truncated below at $-\log 2$.

With `geometry = "unit"`, all d_l incoming coordinates of unit k share the scalar λ_{kl}. There are K_l units per layer, so the earlier specification uses $1/K_l$ as the G/GL half-Cauchy scale and $1 + K_l/\xi_l$ as the GLT inverse-gamma shape.

All three models keep the chosen hidden widths and use a separate scale for the final affine layer. Its weights and intercept have independent zero-centered Gaussian priors conditional on a shared standard deviation κ, which has a half-Cauchy prior with scale 5.

For classification, the conditional prior on each incoming weight or bias has standard deviation $\lambda_{kl,j}$ in the default geometry. The classification likelihood has no observation-variance parameter. Binary classification uses one affine logit for the probability of the second level. For C categorical levels the last is the reference, with logit zero. Each of the first $C - 1$ levels has an affine logit for its log odds relative to the reference. Softmax converts these logits to class probabilities. All free readout coefficients share the scale κ; the prior is specified on contrasts with the chosen reference class.

The function centers predictors and divides by their training sample standard deviations. Regression also standardizes the response and requires every predictor to vary. Classification omits predictors constant in training and requires at least one varying predictor. Supply the original set of columns at prediction time. The function fits a variational approximation with independent Gaussian weight factors. The entries below are available in `control`.

`init_steps` Number of deterministic-network optimizer steps used to initialize the weight means (default 600).

`outer`, `weight_steps` Number of outer iterations (default 60) and weight updates per iteration (default 10).

`lr`, `grad_clip` Initial learning rate for each variational weight update (default 0.003) and gradient norm cap (default 10).

`max_backtracks` Maximum number of learning-rate halvings per weight update (default 10).

`logsd_bounds` Lower and upper bounds on the log standard deviations of the variational weight factors (default `c(-15, 3)`). At both endpoints, `exp(2 * bound)` must give a positive finite variance in double precision. Fitting starts at `log(0.01)` or at the nearest bound if that value falls outside the interval.

`weight_draws`, `noise_draws` Monte Carlo draws per weight update (default 4) and per Gaussian observation-variance update (default 16). Classification uses only the weight draws in this pair.

`monitor_draws`, `endpoint_draws` Monte Carlo draws for the evidence lower bound (ELBO) recorded during training (default 64) and at the independently sampled endpoint (default 256).

With the Jeffreys prior, the objective uses the unnormalized joint density with the arbitrary additive constant of the log prior set to zero. Under this convention the endpoint value is a diagnostic for the variational fit. Normalized model evidence requires a proper prior. The inverse-gamma option includes the normalized log prior in the objective. Its component `noise_neg_kl` combines the expected log prior and factor entropy as minus the KL divergence from the fitted variance factor to its prior. The Jeffreys fit records `noise_prior` and `noise_entropy` separately. Each of the four training draw counts must be an even integer of at least two because the estimator pairs each draw with its negative. With just one pair, the Monte Carlo standard error is undefined. Training uses the requested iteration budget.

Value

An object of class `gltbnn_fit`. Fields include `outcome`, `geometry`, `prior`, `hidden`, the training dimensions `n` and `p`, `predictor_names`, the training centers and scales in `preprocess`, `control`, the chosen observation-variance prior in `noise_prior`, and `seed`. Classifiers store the class order in `levels` and the indices of retained predictors in `preprocess$x_keep`. The `engine` list contains the variational weight parameters (`parameters`), scale factors (`state`), iteration history (`trace`), and the final Monte Carlo ELBO estimate (`endpoint$value`). Use `gltbnn_shrinkage` for coefficient and layer summaries in the paper's notation. In the stored factors, `engine$state$g[[l]]$tau` represents τ_l^2 under G and GL, and τ_l under GLT. For GLT, `g[[l]]$tail$r` is the variational expectation of $1/\xi_l$ and `g[[l]]$tail$eta_mean` is the expectation of $\log \xi_l$. Inverse-gamma factors store their shape in `a` and their scale in `b`. The reciprocal has a gamma distribution with shape `a` and rate `b`. The print method returns its input invisibly.

See Also

`gltbnn_shrinkage`, `predict.gltbnn_fit`

Examples

```
x <- data.frame(z = seq(-1, 1, length.out = 21))
y <- x$z^2 + 0.1 * cos(9 * x$z)
# A short run to illustrate the interface.
fit <- gltbnn_fit(x, y, hidden = 3L, prior = "GL",
                  control = list(init_steps = 10L, outer = 2L, weight_steps = 2L))
fit
new_x <- data.frame(z = c(-0.5, 0, 0.5))
predict(fit, new_x)
predict(fit, new_x, type = "prediction_interval", level = 0.9)

# Probability of each species, with species labels retained in the columns.
flowers <- datasets::iris
held_out <- seq(5, nrow(flowers), by = 5)
species_fit <- gltbnn_fit(flowers[-held_out, 1:4], flowers$Species[-held_out],
```

```
                          outcome = "categorical", hidden = 4L, prior = "GL",
                          control = list(init_steps = 10L, outer = 2L,
                                         weight_steps = 2L))
head(predict(species_fit, flowers[held_out, 1:4], type = "prob", draws = 32L))
```

`gltbnn_shrinkage` *Inspect the shrinkage of hidden weights and biases*

Description

Summarize the conditional prior standard deviation for each hidden coefficient. The variational distribution gives a fitted distribution for its scale; the geometric mean summarizes the typical prior standard deviation.

Usage

```
gltbnn_shrinkage(fit)
```

Arguments

`fit`	A model returned by `gltbnn_fit`.

Details

In the default geometry, each component $w_{kl,j}$ of the incoming vector $\mathbf{w}_{kl}$ has conditional prior variance $\sigma^2\lambda^2_{kl,j}$ in regression and $\lambda^2_{kl,j}$ in classification. A small fitted scale concentrates that coefficient's conditional prior near zero. Under G all coefficients in a layer share τ_l. Under GL the scale is the product of the layer scale and the coefficient's local multiplier. Under GLT the coefficient scales follow a generalized Pareto distribution given the layer scale and tail shape. In the unit geometry, use the unit's scalar λ_{kl} for every coordinate in that unit.

Value

A data frame ordered by hidden layer, receiving unit and incoming coordinate.

`layer`, `unit` One-based indices of the hidden layer l and receiving unit k.

`coordinate`, `is_bias` For the default coefficient geometry, the one-based incoming coordinate j and a logical indicator of the bias. The last coordinate $d_l = K_{l-1} + 1$ multiplies the constant input 1.

`group_size` Number of coefficients represented by the row. This is one for coefficient geometry. For a fit with `geometry = "unit"` or a saved fit from the original package release, the result has one row per hidden unit, with d_l coordinates in each group and the original set of columns.

`scale_precision` Expected inverse squared scale $E_q[\lambda^{-2}_{kl,j}]$ under the fitted variational distribution q. In Gaussian regression, multiply by $E_q[\sigma^{-2}]$ for the expected conditional prior precision.

`geometric_prior_sd` For continuous outcomes, the geometric mean $\exp\{E_q[\log(\sigma\lambda_{kl,j})]\}$ in standardized fitting units. For binary or categorical outcomes this is $\exp\{E_q[\log \lambda_{kl,j}]\}$.

`geometric_layer_scale` Geometric mean of the layer scale, $\exp\{E_q[\log \tau_l]\}$.

`geometric_tail_shape` For GLT, the geometric mean of the layer tail shape, $\exp\{E_q[\log \xi_l]\}$. G and GL return `NA_real_`.

Both layer summaries repeat on every row from the same layer.

See Also

gltbnn_fit

Examples

```
x <- data.frame(z = seq(-1, 1, length.out = 15))
y <- x$z^2
fit <- gltbnn_fit(x, y, hidden = 3L, prior = "GLT",
                  control = list(init_steps = 10L, outer = 2L, weight_steps = 2L))
weights <- gltbnn_shrinkage(fit)
weights[, c("unit", "coordinate", "is_bias", "geometric_prior_sd")]
unique(weights[c("layer", "geometric_layer_scale", "geometric_tail_shape")])
```

---

predict.gltbnn_fit *Predict responses or class probabilities*

Description

Use draws from a fitted network's variational posterior to predict at new observations. Regression predictions use the original response units. Classification predictions retain the training labels and class order.

Usage

```
## S3 method for class 'gltbnn_fit'
predict(object, newdata,
       type = NULL,
       draws = 256L, level = 0.95, seed = 1L, ...)
```

Arguments

object	A model from gltbnn_fit.
newdata	Numeric matrix or numeric data frame with one row per new observation. Supply the same predictors as in training. When both data sets have column names, predictors are matched by name. Otherwise supply them in training order. Prediction reports an error if applying the training standardization or evaluating the predictions exceeds the finite numerical range.
type	For regression, the default "mean" averages draws of the conditional mean. "mean_draws" returns those draws. "response_draws" samples future responses including Gaussian observation noise. "mean_interval" and "prediction_interval" return equal-tailed pointwise intervals for the conditional mean and a future response, respectively. For classification, the default "class" gives the class with the largest posterior mean probability, breaking ties in training level order. Use "prob" for those mean probabilities and "prob_draws" for class probabilities under each sampled network.
draws	Positive integer number of posterior draws. At least two are required for intervals.
level	Probability within each pointwise interval, strictly between zero and one.
seed	Nonnegative integer random seed. The previous .Random.seed is restored after prediction.
...	Reserved for future use.

Details

In each draw the same sampled network evaluates all rows of `newdata`. In regression, a mean draw is its network output. For a response draw, the method also samples a noise variance from its variational factor and adds independent Gaussian errors at the new observations, conditional on that variance. Intervals use sample quantiles of the corresponding draws.

For classification, each network draw yields a vector of probabilities per observation by applying the logistic or softmax link to its affine outputs. The `"prob"` result averages the resulting probabilities over draws. For binary outcomes the second level is the event modeled by the logit.

Value

For `"mean"`, a numeric vector with one entry per observation. For regression draws, a numeric matrix with one row per observation and one column per draw. For intervals, a two-column matrix named `lower` and `upper`. For `"class"`, a factor with the training levels. For `"prob"`, a matrix with one row per observation and one named column per class. For `"prob_draws"`, an array indexed by observation, class, and draw.

See Also

`gltbnn_fit` for a fitting and prediction example.

Index

gltbnn_fit, 2, *5–7*
gltbnn_shrinkage, *4*, 5

predict.gltbnn_fit, *4*, 6
print.gltbnn_fit *(*gltbnn_fit*)*, 2